

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчёт №4

Алгоритмы на графах

(по дисциплине «Алгоритмы и структуры данных»)

Выполнил:

Лабин Макар Андреевич,
АиСД 3.2, Р3231, 466449.

Проверил:

Смирнов Виктор Игоревич

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Яндекс.Контест	1
1.1	Цивилизация	1
1.1.1	Описание решения	1
1.1.2	Асимптотическая сложность	1
1.1.3	Листинг кода	1
1.2	Свинки-копилки	8
1.2.1	Описание решения	8
1.2.2	Асимптотическая сложность	8
1.2.3	Листинг кода	9
1.3	Долой списывание!	10
1.3.1	Описание решения	10
1.3.2	Асимптотическая сложность	11
1.3.3	Листинг кода	11
1.4	Авиаперелёты	12
1.4.1	Описание решения	12
1.4.2	Асимптотическая сложность	13
1.4.3	Листинг кода	13
2	Timus	15
2.1	Network	15
2.1.1	Описание решения	15
2.1.2	Асимптотическая сложность	15
2.1.3	Листинг кода	15
2.2	Mobile Telegraphs	17
2.2.1	Описание решения	17
2.2.2	Асимптотическая сложность	18
2.2.3	Листинг кода	18

1 Яндекс.Контест

1.1 Цивилизация

1.1.1 Описание решения

В задаче требуется найти кратчайший путь между двумя вершинами. Для этого удобно ввести структуру графа $G = (V, E)$, где $v \in V$ — отдельный квадрат на карте мира, а рёбра между ними строятся по принципу соседства в прямоугольной сетке.

Воспользуемся реализацией алгоритма Дейкстры с использованием очереди с приоритетами. Этот алгоритм рассчитывает кратчайшие расстояния до всех вершин от стартовой. Чтобы построить кратчайший путь до конечной вершины в явном виде, нам достаточно хранить сертификат решения — указатель на прошлую вершину, которая обеспечила релаксацию пути.

В конце работы алгоритма необходимо инвертировать полученную последовательность, и ответ будет получен.

1.1.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(M \cdot \log N)$. Поиск текущей минимальной вершины с точки зрения расстояния происходит за логарифмическое время, причём таких операций не превышает количество рёбер в графе, связанных со стартовой вершиной — то есть всех в силу построения графа по прямоугольной сетке. Остальные операции выполняются за константное время.

Пространственная сложность решения — $\mathcal{O}(N \cdot M)$. В памяти хранится информация с входной матрицы, пускай и преобразованная в несколько иную форму при помощи структур и итераторов. Остальные операции вносят менее существенный вклад в потребление памяти.

1.1.3 Листинг кода

```
1 #include <algorithm>
2 #include <array>
3 #include <cstdint>
4 #include <iostream>
5 #include <queue>
6 #include <string>
7 #include <vector>
8
9 struct CostMeta {
10     bool blocked = false;
11     std::size_t value = 0;
12 };
13
14 struct SumMeta {
15     bool is_infinity = true;
16     std::size_t distance = 0;
17 }
```

```

18     bool operator<(const SumMeta& other) const {
19         return !this->is_infinity && (other.is_infinity || this->
20             distance < other.distance);
21     }
22 };
23 struct Point {
24     std::size_t x;
25     std::size_t y;
26 };
27
28 struct VertexMeta {
29     CostMeta local;
30     SumMeta total;
31     bool visited = false;
32     bool has_prev = false;
33     Point prev = {0, 0};
34 };
35
36 class PointNeighborIterator {
37 public:
38     using iterator_category = std::forward_iterator_tag;
39     using value_type = Point;
40     using difference_type = std::ptrdiff_t;
41     using pointer = const Point*;
42     using reference = Point;
43
44     PointNeighborIterator(Point center, std::size_t width, std:::
45         size_t height, std::size_t index = 0)
46         : center_(center), width_(width), height_(height), index_(
47             index) {
48         AdvanceToValid();
49     }
50
51     reference operator*() const {
52         static constexpr std::array<int, 4> KDx = {0, 0, -1, 1};
53         static constexpr std::array<int, 4> KDy = {-1, 1, 0, 0};
54         return {
55             center_.x + static_cast<std::size_t>(KDx.at(index_)),
56             center_.y + static_cast<std::size_t>(KDy.at(index_))
57         };
58     }
59
60     PointNeighborIterator& operator++() {
61         ++index_;
62         AdvanceToValid();
63         return *this;
64     }
65
66     PointNeighborIterator operator++(int) {
67         PointNeighborIterator tmp = *this;

```

```

66     ++(*this);
67     return tmp;
68 }
69
70 friend bool operator==(const PointNeighborIterator& lhs, const
    PointNeighborIterator& rhs) {
71     return lhs.center_.x == rhs.center_.x && lhs.center_.y ==
        rhs.center_.y &&
72         lhs.width_ == rhs.width_ && lhs.height_ == rhs.
            height_ && lhs.index_ == rhs.index_;
73 }
74
75 friend bool operator!=(const PointNeighborIterator& lhs, const
    PointNeighborIterator& rhs) {
76     return !(lhs == rhs);
77 }
78
79 [[nodiscard]] PointNeighborIterator begin() const {
80     return {center_, width_, height_, 0};
81 }
82
83 [[nodiscard]] PointNeighborIterator end() const {
84     return {center_, width_, height_, 4};
85 }
86
87 private:
88     void AdvanceToValid() {
89         static constexpr std::array<int, 4> KDx = {0, 0, -1, 1};
90         static constexpr std::array<int, 4> KDy = {-1, 1, 0, 0};
91         while (index_ < 4) {
92             const int neigh_x = static_cast<int>(center_.x) + KDx.at(
                index_);
93             const int neigh_y = static_cast<int>(center_.y) + KDy.at(
                index_);
94             if (neigh_x >= 0 && static_cast<std::size_t>(neigh_x) <
                width_ && neigh_y >= 0 &&
95                 static_cast<std::size_t>(neigh_y) < height_) {
96                 return;
97             }
98             ++index_;
99         }
100     }
101
102     Point center_;
103     std::size_t width_;
104     std::size_t height_;
105     std::size_t index_;
106 };
107
108 class RectangularGraph {
109 public:

```

```

110 RectangularGraph(std::vector<std::vector<char>>& matrix, Point
    start, Point end)
111     : end_(end), start_(start) {
112     matrix_.resize(matrix.size());
113     for (std::size_t i = 0; i < matrix.size(); i++) {
114         matrix_[i].reserve(matrix[i].size());
115         for (std::size_t j = 0; j < matrix[i].size(); j++) {
116             VertexMeta vmeta{};
117             vmeta.local = GetCost(matrix[i][j]);
118             matrix_[i].push_back(vmeta);
119         }
120     }
121     matrix_[start.y][start.x].total.is_infinity = false;
122 }
123
124 static CostMeta GetCost(char vertex) {
125     switch (vertex) {
126         case '.':
127             return {false, 1};
128         case 'W':
129             return {false, 2};
130         default:
131             return {true, 0};
132     }
133 }
134
135 [[nodiscard]] const VertexMeta& GetVertex(Point point) const {
136     return matrix_[point.y][point.x];
137 }
138
139 const VertexMeta& CalculateShortestPath() {
140     struct QueueItem {
141         std::size_t distance;
142         Point point;
143         bool operator>(const QueueItem& other) const {
144             return distance > other.distance;
145         }
146     };
147
148     std::priority_queue<QueueItem, std::vector<QueueItem>, std::
        greater<QueueItem>> pq;
149     pq.push({0, start_});
150
151     while (!pq.empty()) {
152         auto [dist, vertex_min] = pq.top();
153         pq.pop();
154
155         VertexMeta& vertex_min_meta = this->matrix_[vertex_min.y][
            vertex_min.x];
156

```

```

157     if (vertex_min_meta.visited || dist > vertex_min_meta.
158         total.distance) {
159         continue;
160     }
161
162     vertex_min_meta.visited = true;
163
164     if (vertex_min.x == end_.x && vertex_min.y == end_.y) {
165         break;
166     }
167
168     for (auto edge :
169         PointNeighborIterator(vertex_min, this->matrix_[0].
170             size(), this->matrix_.size())) {
171         VertexMeta& edge_vertex = this->matrix_[edge.y][edge.x];
172         if (edge_vertex.local.blocked) {
173             continue;
174         }
175
176         const std::size_t new_dist = vertex_min_meta.total.
177             distance + edge_vertex.local.value;
178         if (edge_vertex.total.is_infinity || new_dist <
179             edge_vertex.total.distance) {
180             edge_vertex.total.is_infinity = false;
181             edge_vertex.total.distance = new_dist;
182             edge_vertex.has_prev = true;
183             edge_vertex.prev = vertex_min;
184             pq.push({new_dist, edge});
185         }
186     }
187
188     return this->matrix_[this->end_.y][this->end_.x];
189 }
190
191 class PointIterator {
192 public:
193     using iterator_category = std::forward_iterator_tag;
194     using value_type = Point;
195     using difference_type = std::ptrdiff_t;
196     using pointer = const Point*;
197     using reference = Point;
198
199     explicit PointIterator(
200         const std::vector<std::vector<VertexMeta>>& matrix, std
201             ::size_t row, std::size_t col
202     )
203         : matrix_(&matrix), row_(row), col_(col) {
204     }
205
206     reference operator*() const {
207         return Point{col_, row_};
208     }

```

```

203     }
204
205     PointIterator& operator++() {
206         if (row_ < matrix_>size()) {
207             ++col_;
208             if (col_ >= matrix_>at(row_).size()) {
209                 ++row_;
210                 col_ = 0;
211             }
212         }
213         return *this;
214     }
215
216     PointIterator operator++(int) {
217         PointIterator tmp = *this;
218         ++(*this);
219         return tmp;
220     }
221
222     friend bool operator==(const PointIterator& lhs, const
        PointIterator& rhs) {
223         return lhs.row_ == rhs.row_ && lhs.col_ == rhs.col_;
224     }
225
226     friend bool operator!=(const PointIterator& lhs, const
        PointIterator& rhs) {
227         return !(lhs == rhs);
228     }
229
230 private:
231     const std::vector<std::vector<VertexMeta>>* matrix_;
232     std::size_t row_;
233     std::size_t col_;
234 };
235
236 [[nodiscard]] PointIterator begin() const {
237     return matrix_.empty() ? end() : PointIterator(matrix_, 0,
        0);
238 }
239
240 [[nodiscard]] PointIterator end() const {
241     return PointIterator(matrix_, matrix_.size(), 0);
242 }
243
244 [[nodiscard]] PointIterator iterator() const {
245     return begin();
246 }
247
248 private:
249     std::vector<std::vector<VertexMeta>> matrix_;
250     const Point end_;

```

```

251     const Point start_;
252 };
253
254 int main() {
255     std::size_t nrow_s = 0;
256     std::size_t mcol_s = 0;
257     Point start{};
258     Point end{};
259     std::cin >> nrow_s >> mcol_s >> start.y >> start.x >> end.y >>
        end.x;
260
261     std::vector<std::vector<char>> matrix(nrow_s);
262     for (std::size_t i = 0; i < nrow_s; i++) {
263         std::string line;
264         std::cin >> line;
265         matrix[i].reserve(mcol_s);
266         for (const char type : line) {
267             matrix[i].push_back(type);
268         }
269     }
270
271     RectangularGraph graph(matrix, {start.x - 1, start.y - 1}, {
        end.x - 1, end.y - 1});
272     const VertexMeta shortest_path_meta = graph.
        CalculateShortestPath();
273     if (shortest_path_meta.total.is_infinity) {
274         std::cout << -1;
275         return 0;
276     }
277
278     std::cout << shortest_path_meta.total.distance << '\n';
279
280     Point current = {end.x - 1, end.y - 1};
281     std::vector<char> directions;
282     while (graph.GetVertex(current).has_prev) {
283         const Point prev = graph.GetVertex(current).prev;
284
285         const int diff_y = static_cast<int>(current.y) - static_cast
            <int>(prev.y);
286         const int diff_x = static_cast<int>(current.x) - static_cast
            <int>(prev.x);
287
288         if (diff_y == -1) {
289             directions.push_back('N');
290         } else if (diff_y == 1) {
291             directions.push_back('S');
292         } else if (diff_x == -1) {
293             directions.push_back('W');
294         } else if (diff_x == 1) {
295             directions.push_back('E');
296         }

```

```

297
298     current = prev;
299 }
300
301 std::reverse(directions.begin(), directions.end());
302
303 for (const char dir : directions) {
304     std::cout << dir;
305 }
306 std::cout << '\n';
307 return 0;
308 }

```

1.2 Свинки-копилки

1.2.1 Описание решения

Чтобы определить минимальное число копилочек, которое достаточно разбить, чтобы открыть остальные, введём структуру ориентированного графа $G = (V, E)$ следующим образом. Вершины графа $v \in V$ соответствуют копилкам, а рёбра вида $(u, v) \in E$ эквивалентны связи « u открывается ключом из v ».

Возьмём произвольную вершину и покрасим её в некоторый цвет — так мы отметим, что копилка была нами разбита. Далее найдём всех достижимых соседей вершины поиском в глубину, окрашивая обрабатываемые вершины в тот же выбранный цвет. Если мы изначально выбрали вершину M без входящих рёбер, то нам повезло, и мы закончили обработку данного подграфа с началом в этой вершине.

Но нам могло и не повезти, и на деле входящие рёбра у M всё же были. В таком случае при обработке очередного подграфа окажется, что мы пытаемся перекрасить M в другой цвет — мы эту попытку пресекаем и не учитываем текущий подграф в аккумулируемой переменной, поскольку он является началом уже учтённого подграфа.

Если же в ориентированном графе есть неориентированные циклы, то мы смотрим на цвет окрашиваемой вершины. Если он совпадает с цветом текущего подграфа, то пропускаем эту вершину: эта ветка уже была обработана.

Приведённый выше алгоритм нужно запустить для всех неокрашенных вершин в графе, чтобы в аккумулируемой переменной получить число наибольших ориентированных подграфов. Оно и будет соответствовать минимальному числу копилочек.

1.2.2 Асимптотическая сложность

Временная сложность решения — линейная $O(N)$. Структура графа строится за линейное время относительно числа рёбер. Подсчёт ориентированных подграфов происходит при помощи итеративной реализации поиска в глубину, которая отрабатывает за линейное время относительно числа вершин и рёбер. Чтение и запись происходят за константное время.

Пространственная сложность решения — линейная $O(N)$. В памяти материализуется структура графа, которая хранит для каждой вершины кон-

стантное количество данных. Суммарно число рёбер равно числу вершин, поэтому массивы соседних вершин не разрастаются.

1.2.3 Листинг кода

```
1 #include <cstdint>
2 #include <iostream>
3 #include <list>
4 #include <unordered_map>
5 #include <vector>
6
7 struct Vertex {
8     bool visited = false;
9     std::size_t color = 0;
10    std::vector<std::size_t> neighbours;
11 };
12
13 class Graph {
14 public:
15     void AddEdge(std::size_t from_id, std::size_t to_id) {
16         if (this->vertices_.count(from_id) == 0U) {
17             this->vertices_[from_id] = {false, 0, {}};
18         }
19         if (from_id == to_id) {
20             return;
21         }
22         this->vertices_[from_id].neighbours.push_back(to_id);
23     }
24
25     std::size_t CountSources() {
26         std::size_t sources_count = 0;
27         std::size_t current_color = 0;
28         for (std::size_t i = 1; i <= this->vertices_.size(); i++) {
29             if (this->vertices_[i].visited) {
30                 continue;
31             }
32             std::list<std::size_t> to_visit = {i};
33             std::size_t broken_count = 1;
34             while (!to_visit.empty()) {
35                 Vertex& top_vertex = this->vertices_[to_visit.front()];
36                 to_visit.pop_front();
37                 top_vertex.visited = true;
38                 top_vertex.color = current_color;
39                 for (const auto& neighbour : top_vertex.neighbours) {
40                     const Vertex& neighbour_vertex = this->vertices_[
41                         neighbour];
42                     if (neighbour_vertex.visited && neighbour_vertex.
43                         color != current_color) {
44                         broken_count = 0;
45                         continue;
46                     }
47                 }
48             }
49         }
50     }
51 }
```

```

45         if (neighbour_vertex.visited) {
46             continue;
47         }
48         to_visit.push_front(neighbour);
49     }
50 }
51 sources_count += broken_count;
52 current_color++;
53 }
54 return sources_count;
55 }
56
57 private:
58     std::unordered_map<std::size_t, Vertex> vertices_;
59 };
60
61 int main() {
62     std::size_t n_count = 0;
63     std::cin >> n_count;
64     Graph graph;
65     for (std::size_t i = 1; i <= n_count; i++) {
66         std::size_t open_idx = 0;
67         std::cin >> open_idx;
68         // inverting the direction
69         graph.AddEdge(i, open_idx);
70     }
71     std::cout << graph.CountSources();
72     return 0;
73 }

```

1.3 Долой списывание!

1.3.1 Описание решения

Задачу можно свести к вопросу о возможности построения структуры двудольного графа $G = (V \equiv A \cup B, E)$ на множестве студентов: доля A соответствует списывающим, а другая — B — дающим списывать.

По условию задачи ребра $(u, v) \in E$ неориентированные, поэтому мы вольны распределять студентов в обе категории произвольно, пока сохраняется структура. Для этого возьмём случайную вершину в графе и покрасим её, к примеру, в красный цвет. Определим соседей вершины и для всех них установим другой цвет — чёрный — если они ещё не были покрашены. Так для слоёв вершин будем чередовать цвета, пока не закончатся соседи.

Если в процессе поиска в глубину нашлась уже окрашенная вершина и возник конфликт цветов, то граф невозможно покрасить в два цвета — то есть и структура двудольного графа на нём невозможна без удаления рёбер.

Чтобы учесть случай несвязного графа, пройдемся по всем непокрашенным вершинам в главном цикле.

1.3.2 Асимптотическая сложность

Временная сложность решения — линейная $O(N + M)$. Построение структуры графа занимает линейное время относительно числа рёбер, а поиск в глубину — линейное время относительно числа вершин и рёбер.

Пространственная сложность решения — линейная $O(N + M)$. Для каждой вершины хранится константное число данных, а массивы соседних вершин суммарно содержат не более линейного числа элементов.

1.3.3 Листинг кода

```
1 #include <cstdint>
2 #include <iostream>
3 #include <list>
4 #include <unordered_map>
5 #include <vector>
6
7 struct Vertex {
8     std::size_t color = 0;
9     bool visited = false;
10    std::vector<std::size_t> neighbours;
11 };
12
13 class Graph {
14 public:
15     void AddEdge(std::size_t from_id, std::size_t to_id) {
16         if (this->vertices_.count(from_id) == 0U) {
17             this->vertices_[from_id] = {0, false, {}};
18         }
19         if (this->vertices_.count(to_id) == 0U) {
20             this->vertices_[to_id] = {0, false, {}};
21         }
22         this->vertices_[from_id].neighbours.push_back(to_id);
23         this->vertices_[to_id].neighbours.push_back(from_id);
24     }
25
26     static std::size_t SwitchColor(std::size_t from_color) {
27         return from_color == 0 ? 1 : 0;
28     }
29
30     bool TryColor() {
31         bool is_possible = true;
32         for (std::size_t i = 1; i <= this->vertices_.size(); i++) {
33             if (this->vertices_[i].visited) {
34                 continue;
35             }
36             std::list<std::size_t> to_visit = {i};
37             while (!to_visit.empty()) {
38                 Vertex& top_vertex = this->vertices_[to_visit.front()];
39                 to_visit.pop_front();
40                 top_vertex.visited = true;
```

```

41     for (const auto& neighbour : top_vertex.neighbours) {
42         Vertex& neighbour_vertex = this->vertices_[neighbour
43             ];
44         if (neighbour_vertex.visited && neighbour_vertex.
45             color == top_vertex.color) {
46             is_possible = false;
47             break;
48         }
49         if (neighbour_vertex.visited) {
50             continue;
51         }
52         neighbour_vertex.color = Graph::SwitchColor(
53             top_vertex.color);
54         to_visit.push_front(neighbour);
55     }
56 }
57
58 private:
59     std::unordered_map<std::size_t, Vertex> vertices_;
60 };
61
62 int main() {
63     std::size_t n_students = 0;
64     std::size_t m_pairs = 0;
65     std::cin >> n_students >> m_pairs;
66     Graph graph;
67     for (std::size_t i = 0; i < m_pairs; i++) {
68         std::size_t from_id = 0;
69         std::size_t to_id = 0;
70         std::cin >> from_id >> to_id;
71         graph.AddEdge(from_id, to_id);
72     }
73     std::cout << (graph.TryColor() ? "YES" : "NO");
74     return 0;
75 }

```

1.4 Авиаперелёты

1.4.1 Описание решения

Во входных данных задачи даётся матрица весов, которой можно задать полноценный граф. Чтобы определить величину максимального ребра на минимальном пути между двумя городами (*с точки зрения суммарного расстояния*) воспользуемся модификацией алгоритма Флойда-Уоршелла.

Заменим метрику суммарного расстояния величиной ребра с максимальным весом: это возможно поскольку обе операции ассоциативны, то есть могут быть применены в разной группировке одного пути. Алгоритм перебирает все

доступные варианты путей, поэтому гарантированно минимизирует указанную метрику.

Ответ к задаче — максимум в матрице посчитанных метрик.

1.4.2 Асимптотическая сложность

Временная сложность решения — кубическая $\mathcal{O}(N^3)$. Алгоритм перебирает все возможные тройки вершин и считает метрики для полученных путей за константное время. Определение максимума в матрице происходит за квадратичное время.

Пространственная сложность решения — квадратичная $\mathcal{O}(N^2)$. В алгоритме материализована матрица весов, которая переиспользуется для подсчёта матрицы метрик.

1.4.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <iostream>
4 #include <vector>
5
6 class Graph {
7 public:
8     explicit Graph(std::size_t n_count)
9         : distances_(n_count + 1, std::vector<std::size_t>(n_count
10             + 1, 0)) {
11     }
12     void AddEdge(std::size_t from_id, std::size_t to_id, std::
13         size_t weight) {
14         this->distances_[from_id][to_id] = weight;
15     }
16     std::size_t FindExpensiveEdge(std::size_t n_count) {
17         for (std::size_t third_vertex = 1; third_vertex <= n_count;
18             third_vertex++) {
19             for (std::size_t first_vertex = 1; first_vertex <= n_count
20                 ; first_vertex++) {
21                 for (std::size_t second_vertex = 1; second_vertex <=
22                     n_count; second_vertex++) {
23                     const std::size_t other_path = std::max(
24                         this->distances_[first_vertex][third_vertex],
25                         this->distances_[third_vertex][second_vertex]
26                     );
27                     this->distances_[first_vertex][second_vertex] =
28                         std::min(this->distances_[first_vertex][
29                             second_vertex], other_path);
30                 }
31             }
32         }
33         std::size_t expensive_edge = 0;
34         for (std::size_t first_vertex = 1; first_vertex <= n_count;
35             first_vertex++) {
```

```

29     for (std::size_t second_vertex = 1; second_vertex <=
        n_count; second_vertex++) {
30         expensive_edge = std::max(expensive_edge, this->
            distances_[first_vertex][second_vertex]);
31     }
32 }
33 return expensive_edge;
34 }
35
36 private:
37     std::vector<std::vector<std::size_t>> distances_;
38 };
39
40 int main() {
41     std::size_t n_count = 0;
42     std::cin >> n_count;
43     Graph graph(n_count);
44     for (std::size_t row_idx = 0; row_idx < n_count; row_idx++) {
45         for (std::size_t col_idx = 0; col_idx < n_count; col_idx++)
46         {
47             std::size_t weight = 0;
48             std::cin >> weight;
49             graph.AddEdge(row_idx + 1, col_idx + 1, weight);
50         }
51     }
52     std::cout << graph.FindExpensiveEdge(n_count);
53     return 0;
54 }

```

2 Timus

2.1 Network

2.1.1 Описание решения

Задачу можно свести к составлению минимального остовного дерева: оно обеспечивает связность всех хабов. Кроме того, есть доказанное утверждение, что всякое минимальное остовное дерево является минимально узким — это значит, что размер максимального ребра в остовном дереве минимален (*что и требуется по условию задачи*).

Для реализации воспользуемся алгоритмом Краскала с применением системы непересекающихся множеств — последнее позволяет формулировать поддеревья в графе как классы эквивалентности и объединять их за логарифмическое время.

2.1.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(M \cdot \log M + N)$. В задаче применяется быстрая сортировка массива рёбер, и инициализируется СНМ на множестве вершин графа. Алгоритм Краскала проходится по всем рёбрам и объединяет классы эквивалентности почти за константное время, что не превышает сложность предыдущих операций.

Пространственная сложность решения — линейное $\mathcal{O}(N + M)$. В задаче материализованы все рёбра графа и вершины, остальное требует константного количества памяти.

2.1.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdlib>
3 #include <iostream>
4 #include <vector>
5
6 class DSU {
7 public:
8     explicit DSU(std::size_t n) {
9         this->parent_.resize(n + 1);
10        this->rank_.resize(n + 1, 0);
11        for (std::size_t i = 0; i <= n; ++i) {
12            this->parent_[i] = i;
13        }
14    }
15
16    std::size_t Find(std::size_t vertex) {
17        if (vertex == this->parent_[vertex]) {
18            return vertex;
19        }
20        return this->parent_[vertex] = Find(this->parent_[vertex]);
21    }
22 }
```

```

23 bool Union(std::size_t left, std::size_t right) {
24     left = Find(left);
25     right = Find(right);
26     if (left != right) {
27         if (this->rank_[left] < this->rank_[right]) {
28             std::swap(left, right);
29         }
30         this->parent_[right] = left;
31         if (this->rank_[left] == this->rank_[right]) {
32             this->rank_[left]++;
33         }
34         return true;
35     }
36     return false;
37 }
38
39 private:
40     std::vector<std::size_t> parent_;
41     std::vector<std::size_t> rank_;
42 };
43
44 struct Edge {
45     std::size_t from_vertex;
46     std::size_t to_vertex;
47     std::size_t weight;
48
49     bool operator<(const Edge& other) const {
50         return this->weight < other.weight;
51     }
52 };
53
54 class Graph {
55 public:
56     explicit Graph(std::size_t n_vertices) : n_vertices_(
57         n_vertices) {
58
59     void AddEdge(std::size_t from_vertex, std::size_t to_vertex,
60                 std::size_t weight) {
61         edges_.push_back({from_vertex, to_vertex, weight});
62     }
63
64     std::vector<Edge> CreatePlan() {
65         std::vector<Edge> spanning_list;
66         std::sort(this->edges_.begin(), this->edges_.end());
67         DSU dsu(n_vertices_);
68         for (const auto& edge : edges_) {
69             if (dsu.Union(edge.from_vertex, edge.to_vertex)) {
70                 spanning_list.push_back(edge);
71             }
72         }
73     }
74 };

```

```

72     return spanning_list;
73 }
74
75 private:
76     std::size_t n_vertices_;
77     std::vector<Edge> edges_;
78 };
79
80 int main() {
81     std::size_t n_networks = 0;
82     std::size_t m_connections = 0;
83     std::cin >> n_networks >> m_connections;
84     Graph graph(n_networks);
85     for (std::size_t i = 0; i < m_connections; i++) {
86         std::size_t from_vertex = 0;
87         std::size_t to_vertex = 0;
88         std::size_t weight = 0;
89         std::cin >> from_vertex >> to_vertex >> weight;
90         graph.AddEdge(from_vertex, to_vertex, weight);
91     }
92     std::vector<Edge> plan = graph.CreatePlan();
93     auto max_edge = std::max_element(plan.begin(), plan.end());
94     std::cout << max_edge->weight << '\n';
95     std::cout << plan.size() << '\n';
96     for (const auto& edge : plan) {
97         std::cout << edge.from_vertex << " " << edge.to_vertex << '\n';
98     }
99     return 0;
100 }

```

2.2 Mobile Telegraphs

2.2.1 Описание решения

Задача легко решается алгоритмом Дейкстры, если преобразовать входные данные в ориентированный взвешенный граф $G = (V, E)$, где $v \in V$ — бойцы на поле с телеграфами, а $(u, v) \in E$ — связь между телеграфами u и v .

По условию задачи, две вершины в графе связаны, если их строковые номера различаются:

- одной цифрой;
- двумя цифрами, которые можно поменять местами.

При этом вес ребра зависит от длины общего префикса и определяется массивом из входных данных.

Восстановление сертификата пути происходит за линейное время путём перехода по указателю на предыдущую вершину, которая участвовала в релаксации пути.

2.2.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(N \cdot \log N + M \cdot \log N)$. Граф инициализируется с применением оптимизации на базе знания структуры входных строк: если проверять все пары строк и проверять условие, то решение быстро упрётся во временные рамки. Поэтому сортируем входной массив в лексикографическом порядке и ищем похожие строки при помощи бинарного поиска.

Также используется алгоритм Дейкстры с применением очереди с приоритетами, который проходится по всем достижимым рёбрам графа из стартовой вершины и определяет минимальных соседей за логарифмическое время.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Все вершины материализованы в графе, а число рёбер для каждой вершины не более чем константно из-за специфики построения графа по строкам.

2.2.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <iostream>
4 #include <limits>
5 #include <queue>
6 #include <stdexcept>
7 #include <vector>
8
9 namespace {
10 const std::size_t STRING_SIZE = 10;
11 const std::size_t INF = std::numeric_limits<std::size_t>::max();
12 } // namespace
13
14 struct AdjacentMeta {
15     bool are_adjacent = false;
16     std::size_t common_prefix = 0;
17 };
18
19 struct Edge {
20     std::size_t to_idx;
21     std::size_t weight;
22 };
23
24 struct Vertex {
25     std::size_t id;
26     Vertex* prev = nullptr;
27     std::vector<Edge> neighbours;
28 };
29
30 struct DijkstraResult {
31     std::size_t dist;
32     Vertex* end;
33 };
34
35 struct SearchNode {
```

```

36     std::string str;
37     std::size_t id;
38     bool operator<(const SearchNode& other) const {
39         return str < other.str;
40     }
41 };
42
43 class Graph {
44 public:
45     Graph(const std::vector<std::string>& strings, const std::
        vector<std::size_t>& costs)
46         : vertices_(std::vector<Vertex>(strings.size() + 1)) {
47         for (std::size_t i = 0; i < strings.size(); ++i) {
48             vertices_[i].id = i;
49         }
50
51         std::vector<SearchNode> search_array(strings.size());
52         for (std::size_t i = 0; i < strings.size(); ++i) {
53             search_array[i] = {strings[i], i};
54         }
55         std::sort(search_array.begin(), search_array.end());
56
57         auto find_id = [search_array](const std::string& target_str)
            -> std::size_t {
58             SearchNode target{target_str, 0};
59             auto it = std::lower_bound(search_array.begin(),
                search_array.end(), target);
60             if (it != search_array.end() && it->str == target_str) {
61                 return it->id;
62             }
63             return INF;
64         };
65
66         for (std::size_t u = 0; u < strings.size(); ++u) {
67             std::string s = strings[u];
68             for (std::size_t i = 0; i < STRING_SIZE; ++i) {
69                 const char original_char = s[i];
70                 for (char d = '0'; d <= '9'; ++d) {
71                     if (d == original_char) {
72                         continue;
73                     }
74                     s[i] = d;
75                     std::size_t v = find_id(s);
76                     if (v != INF) {
77                         vertices_[u].neighbours.push_back({v, costs[i]});
78                     }
79                 }
80                 s[i] = original_char;
81             }
82
83             for (std::size_t i = 0; i < STRING_SIZE; ++i) {

```

```

84     for (std::size_t j = i + 1; j < STRING_SIZE; ++j) {
85         if (s[i] == s[j]) {
86             continue;
87         }
88
89         std::swap(s[i], s[j]);
90         std::size_t v = find_id(s);
91         if (v != INF) {
92             vertices_[u].neighbours.push_back({v, costs[i]});
93         }
94         std::swap(s[i], s[j]);
95     }
96 }
97 }
98 }
99
100 static const AdjacentMeta AreAdjacent(std::string_view left,
101     std::string_view right) {
102     if (left.size() != right.size()) {
103         throw std::invalid_argument("strings should have equal
104             length");
105     }
106     std::size_t left_idx = 0;
107     AdjacentMeta meta;
108     while (left_idx < STRING_SIZE && left[left_idx] == right[
109         left_idx]) {
110         left_idx++;
111         meta.common_prefix++;
112     }
113     std::size_t right_idx = left_idx + 1;
114     while (right_idx < STRING_SIZE && left[right_idx] == right[
115         right_idx]) {
116         right_idx++;
117     }
118     std::size_t third_idx = right_idx + 1;
119     while (third_idx < STRING_SIZE && left[third_idx] == right[
120         third_idx]) {
121         third_idx++;
122     }
123     meta.are_adjacent = (left_idx == STRING_SIZE) || (right_idx
124         == STRING_SIZE) ||
125         (third_idx == STRING_SIZE && left[
126             left_idx] == right[right_idx] &&
127             left[right_idx] == right[left_idx]);
128     return meta;
129 }
130
131 DijkstraResult Dijkstra(std::size_t start_idx, std::size_t
132     end_idx) {
133     std::vector<std::size_t> dist(vertices_.size(), INF);
134     std::priority_queue<

```

```

127         std::pair<std::size_t, std::size_t>,
128         std::vector<std::pair<std::size_t, std::size_t>>,
129         std::greater<std::pair<std::size_t, std::size_t>>>
130         pq;
131     dist[start_idx] = 0;
132     pq.emplace(0, start_idx);
133     while (!pq.empty()) {
134         auto [current_dist, u_idx] = pq.top();
135         pq.pop();
136         if (u_idx == end_idx) {
137             return {dist[end_idx], &vertices[end_idx]};
138         }
139         if (current_dist > dist[u_idx]) {
140             continue;
141         }
142         for (const auto& edge : vertices[u_idx].neighbours) {
143             const std::size_t v_idx = edge.to_idx;
144             if (dist[u_idx] + edge.weight < dist[v_idx]) {
145                 dist[v_idx] = dist[u_idx] + edge.weight;
146                 vertices[v_idx].prev = &vertices[u_idx];
147                 pq.emplace(dist[v_idx], v_idx);
148             }
149         }
150     }
151     return {dist[end_idx], &vertices[end_idx]};
152 }
153
154 private:
155     std::vector<Vertex> vertices_;
156 };
157
158 int main() {
159     std::size_t n_count = 0;
160     std::cin >> n_count;
161     std::vector<std::size_t> costs(String_Size);
162     for (std::size_t i = 0; i < String_Size; i++) {
163         std::cin >> costs[i];
164     }
165     std::vector<std::string> strings(n_count);
166     for (std::size_t i = 0; i < n_count; i++) {
167         std::cin >> strings[i];
168     }
169     Graph graph(strings, costs);
170     const DijkstraResult result = graph.Dijkstra(0, n_count - 1);
171     if (result.dist == INF) {
172         std::cout << -1;
173         return 0;
174     }
175     std::vector<std::size_t> path;
176     const Vertex* curr = result.end;
177     std::size_t count = 0;

```

```
178 while (curr != nullptr) {
179     path.push_back(curr->id);
180     curr = curr->prev;
181     count++;
182 }
183 std::reverse(path.begin(), path.end());
184 std::cout << result.dist << '\n';
185 std::cout << count << '\n';
186 for (const std::size_t vertex_id : path) {
187     std::cout << vertex_id + 1 << " ";
188 }
189 return 0;
190 }
```